

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МОЭВМ

ЛАБОРАТОРНАЯ РАБОТА №4
по дисциплине «Объектно-ориентированное программирование»
Тема: Создание игры в стиле ООП

Студентка гр. 4342

Киселева А.Д.

Преподаватель

Жангиров Т.Р.

Санкт-Петербург

2025

ВВЕДЕНИЕ

Цель: разработка модульной и расширяемой архитектуры пошаговой тактической игры с применением современных принципов объектно-ориентированного программирования на C++, обеспечивающей гибкость модификации игровой логики и эффективное управление ресурсами.

Задачи:

1. Создать класс считывающий ввод пользователя и преобразующий ввод пользователя в объект команды;
2. Создать класс отрисовки игры;
3. Создать шаблонный класс управления игрой, которому в качестве параметра шаблона будет передаваться класс считывания и преобразования ввода;
4. Создать шаблонный класс визуализации игры, которому в качестве шаблона будет передаваться класс отрисовки игры.

ОПИСАНИЕ СТРУКТУРЫ КОДА

Класс *ProcessInput* отвечает за обработку команд при вводе с клавиатуры и мыши.

Поля:

- *CommandType type* — объект перечисления *CommandType*, хранящий текущую команду.
- *std::pair<int, int> clickCoords* — координаты клика мыши.

Методы класса:

1. *ProcessInput()* - конструктор по умолчанию. Присваивает полю типа текущей команды *type* значение *NONE*.
2. *bool processEvent(const sf::Event& event)* — метод обработки события графического интерфейса. Если нажата кнопка на клавиатуре, вызывается функция обработки ввода с клавиатуры, если нажата левая кнопка мыши, изменяется значение текущей команды и в соответствующее поле устанавливаются координаты клика мыши.

Принимаемый аргумент:

const sf::Event& event — ссылка на событие.

Возвращаемое значение:

true, если событие обработано, иначе *false*.

3. *bool processKeyboard(sf::Keyboard::Key key)* — метод обработки ввода с клавиатуры. Устанавливает тип команды в зависимости от нажатой кнопки.

Принимаемый аргумент:

sf::Keyboard::Key key — код нажатой клавиши клавиатуры.

Возвращаемое значение:

true, если клавиша одна из допустимых, иначе *false*.

4. *CommandType getCommandType() const* — метод получения типа текущей команды, используется в классе *GameController* для обработки полученных команд.

Возвращаемое значение:

CommandType command — тип текущей команды.

5. *std::pair<int, int> getClickCoords() const* — метод получения координат клика мыши, используется в классе *GameController*.

Возвращаемое значение:

std::pair<int, int> clickCoords — координаты клика.

Шаблонный класс *VisualDisplay* отвечает за тип визуализации игры. Через него осуществляется связь с конкретным классом отрисовки и вызывается метод отображения игрового процесса.

Поля:

- *RenderType renderer* — объект класса-отрисовщика.

Методы:

1. *VisualDisplay()* - конструктор по умолчанию, создает объект класса-отрисовщика.
2. *void render(renderData renderParameters, GameStatus status)* — метод отрисовки игры. Вызывает одноименный метод у объекта класса-отрисовщика.

Принимаемые аргументы:

renderData renderParameters — структура, содержащая все нужные данные для отображения.

GameStatus status — текущее состояние игры.

Структура *renderData* включает в себя данные, необходимые для отображения игры. Объект создается и инициализируется в классе *GameController*, обновляется по мере обработки определенных команд пользователя.

Поля:

- *Player player* — объект игрока;
- *Enemy enemy* — объект врага;
- *Field field* — объект поля;
- *std::pair<int, int> towerCoords* — координаты вражеской башни;
- *int currentMoves* — текущее количество ходов;
- *sf::RenderWindow* window* — указатель на окно отображения игры;
- *LevelAttributes LVLParameters* — структура с параметрами уровня;
- *int selectedSpellCard* — выбранная карточка заклинаний;
- *OverlayType currentOverlayType* — тип заливки для отрисовки площади поражения цели;
- *bool showRangeOverlay* — поле, определяющее показана ли заливка в данный момент.

Метод:

- *renderData()* - конструктор по умолчанию, инициализирует объекты игрока, врага и поля с нулевыми параметрами.

Класс *GameGraphics* отвечает за графическое отображение игры.

Поля:

- *sf::Texture playerTexture* — текстура игрока;
- *sf::Texture enemyTexture* — текстура врага;
- *sf::Texture defaultCellTexture* — текстура обычной клетки поля;
- *sf::Texture obstacleCellTexture* — текстура клетки с препятствием;

- *sf::Texture freezeCellTexture* — текстура клетки заморозки;
- *sf::Texture trapCellTexture* — текстура ловушки;
- *sf::Texture towerTexture* — текстура вражеской башни;
- *sf::Font font* — основной шрифт;
- *sf::Font fontResult* — жирный шрифт для заголовков;
- *sf::Text infoText* — шрифт для текста в правой панели с информацией о текущей игре;
- *sf::Text instructionText* - текст с инструкциями по управлению и целью уровня;
- *sf::Text gameOverText* — заголовок "GAME OVER" при поражении;
- *sf::Text menuTitle* — заголовок главного меню "CAT & MICE";
- *sf::Text levelCompleteTitle* - заголовок "LEVEL COMPLETED!" при победе;
- *sf::Text improveTitle* - заголовок "CHOOSE IMPROVEMENT" в меню улучшений;
- *int CELL_SIZE* — размер клетки поля;
- *int CARD_WIDTH* — длина карточки заклинания;
- *int CARD_HEIGHT* — высота карточки заклинания;
- *int CARD_MARGIN* — рамка карточки заклинания;
- *int CARDS_Y_POSITION* — координата y для центрирования карточек;
- *std::vector<sf::RectangleShape> cardBackgrounds* — прямоугольники для фона карточек заклинаний;
- *std::vector<sf::Text> cardTitles* — тексты названий заклинаний на карточках;
- *std::vector<sf::Text> cardDescriptions* — тексты описаний заклинаний на карточках;
- *std::vector<sf::RectangleShape> rangeOverlay* — прямоугольники для заливки зоны действия атаки/заклинания;

- `std::vector<sf::RectangleShape> menuButtons` — кнопки главного меню;
- `std::vector<sf::Text> menuButtonTexts` — тексты кнопок главного меню;
- `std::vector<sf::RectangleShape> levelCompleteButtons` — кнопки экрана победы;
- `std::vector<sf::Text> levelCompleteButtonTexts` — текст кнопок экрана победы;
- `std::vector<sf::RectangleShape> gameOverButtons` — кнопки экрана поражения;
- `std::vector<sf::Text> gameOverButtonTexts` — текст кнопок экрана поражения;
- `std::vector<sf::RectangleShape> improveButtons` — кнопки меню улучшений;
- `std::vector<sf::Text> improveButtonTexts` — текст кнопок меню улучшений;
- `std::vector<sf::Text> improveDescriptions` — описание кнопок меню улучшений;

Методы:

1. `GameGraphics()` - конструктор по умолчанию. Вызывает метод загрузки текстур, устанавливает центрирование по оси *Y* для карточек заклинаний, настраивает параметры текстов правой панели инструкций, меню, победы и поражения.
2. `void loadAssets()` - метод загрузки текстур из файлов.
3. `void render(renderData renderParameters, GameStatus gameStatus)` — метод, который в зависимости от текущего состояния игры определяет, какую отрисовку использовать.

Принимаемые аргументы:

renderData renderParameters — структура с параметрами, которые нужно отобразить;

GameStatus gameStatus — состояние игры.

4. *void initializeSpellCards(renderData renderParameters)* — метод инициализации карточек заклинаний. Создаются серые прямоугольники с белой рамкой, определяется позиция на экране и настраиваются параметры текста каждой карточки.

Принимаемый аргумент:

renderData renderParameters — структура с параметрами для отрисовки.

5. *void update(renderData renderParameters)* — метод обновления отрисовки во время игры. При выборе карточки заклинаний делает ее рамку желтой, устанавливает заголовки и описания доступных игроку карточек, обновляет информацию о типе атаки, возможности движения и целях уровня, отображает статистику игрока и врага.

Принимаемый аргумент:

renderData renderParameters — структура с параметрами для отрисовки.

6. *void renderPlaying(renderData renderParameters)* — метод отрисовки основного игрового процесса. Рисует клетки поля, персонажей, ловушки, бар карточек заклинаний и правую информационную панель.

Принимаемый аргумент:

renderData renderParameters — структура с параметрами для отрисовки.

7. *void renderMainMenu(sf::RenderWindow* wndw)* — метод отрисовки главного меню. Отображаются кнопки, заголовок и фон.

Принимаемый аргумент:

sf::RenderWindow wndw* — указатель на окно, в котором будет происходить отображение.

8. *void renderLevelComplete(renderData renderParameters)* — метод отрисовки экрана победы. Отображается статистика, заголовок, кнопки и фон.

Принимаемый аргумент:

renderData renderParameters — структура с параметрами для отрисовки.

9. *void renderGameOver(renderData renderParameters)* — метод отрисовки экрана поражения. Отображается статистика, заголовок, кнопка возврата в меню и фон.

Принимаемый аргумент:

renderData renderParameters — структура с параметрами для отрисовки.

10. *void renderImprove(renderData renderParameters)* — метод отрисовки экрана улучшения. Отображаются кнопки, заголовок и фон.

Принимаемый аргумент:

renderData renderParameters — структура с параметрами для отрисовки.

11. *void updateCellSize(int newSize)* — метод обновления размера клетки поля. Используется при переходе на следующий уровень, чтобы сохранить исходный размер окна, положение бара карточек и правой информационной панели.

Принимаемый аргумент:

int newSize — новый размер клетки.

12. *void updateRangeOverlay(renderData renderParameters)* — метод, отвечающий за заливку зоны действия атаки/заклинания. В зависимости от типа заливки вызывает методы ее отображения.

Принимаемый аргумент:

renderData renderParameters — структура с параметрами для отрисовки.

13. *void showSpellRange(Player player, Field field, int range)* — метод отображения радиуса действия заклинания.

Принимаемые аргументы:

Player player — объект игрока;

Field field — объект поля;

int range — манхеттенский радиус заклинания.

14. *void showTowerRange(Field field, std::pair<int, int> towerCoords)* — метод отображения площади, на которой наносит урон вражеская башня.

Принимаемые аргументы:

Field field — объект поля;

std::pair<int, int> towerCoords — координаты башни.

15. *void showCombatRange(Player player, Field field)* — метод отображения радиуса атаки игрока.

Принимаемые аргументы:

Player player — объект игрока;

Field field — объект поля.

Шаблонный класс *GameController* обрабатывает команды, поступившие от класса считывания ввода, обновляет параметры отрисовки и запускает игру.

Поля:

- *T inputProcessor* — объект обработчика ввода пользователя;

- *VisualDisplay<GameGraphics> GUI* — объект класса визуального отображения с конкретным классом отрисовки;
- *sf::RenderWindow window* — окно, в котором будет отображаться игра;
- *Game& game* — ссылка на игру;
- *GameStatus gameStatus* — текущее состояние игры;
- *renderData renderParameters* — структура с параметрами для отрисовки;
- *int windowHeight* — ширина окна;
- *int windowHeight* — высота окна;
- *bool waitingForTarget* — поле, показывающее ожидается ли выбор цели для применения заклинания.

Методы:

1. *GameController(Game& currentGame)* — конструктор с параметром. Создает объекты обработчика ввода и графического отображения, устанавливает текущий статус игры на главное меню, создает окно отрисовки и обновляет параметры структуры для отрисовки игры.

Принимаемый аргумент:

Game& currentGame — ссылка на текущую игру.

2. *void processCommand()* - метод обработки команд от класса считывания ввода. В зависимости от полученной команды в игре происходят определенные действия.
3. *void processClick(int x, int y)* — метод обработки команды клика мыши. В зависимости от состояния игры вызываются методы для обработки клика в игре (для заклинаний) или для обработки клика в меню (нажатие графических кнопок).

Принимаемые аргументы:

int x, int y — координаты клика мыши.

4. *void updateRenderAttributes()* - метод обновления структуры с данными для отрисовки.
5. *void startGame()* - метод запуска игры. Пока открыто графическое окно, происходит обработка команд и отрисовка игры.

В функции *main()* создается объект игры и объект класса управления игрой, которому в качестве параметра шаблона подается класс обработки ввода *ProcessInput*. Игра запускается посредством вызова у управляющего объекта метода *startGame()*.

Взаимосвязь классов представлена на UML-диаграмме (см. Приложение).

ЗАКЛЮЧЕНИЕ

В результате лабораторной работы были созданы классы, которые осуществляют взаимодействие пользователя с игрой и отображают ее на экране: шаблонный класс *GameController* является основным управляющим компонентом, связывающим все составляющие системы; шаблонный класс *VisualDisplay* служит «прослойкой» между управляющим классом и конкретным способом отрисовки. Главным преимуществом этих классов является возможность изменения параметров шаблонов, а именно класса графического отображения или класса обработки команд пользователя, что позволяет поменять конкретную реализацию игры, не затрагивая основную логику. На данный момент за обработку команд отвечает класс *ProcessInput*, за отображение — *GameGraphics*. Также соблюдены принципы инкапсуляции, исключено дублирование данных. Программа протестирована и работает корректно.

ПРИЛОЖЕНИЕ

